

需求规格说明书

项目名称： 社交网络平台（Social Network Platform）

项目组组长： 向楚玉

团队成员： 向楚玉、胡紫怡

第1章 引言

1.1 编写目的

本需求规格说明书旨在为“社交网络平台”项目的开发提供完整、准确、清晰的功能定义与需求描述，作为后续系统设计、编码实现、测试验收的根本依据。

本文档的读者对象包括：项目开发团队成员、课程指导老师及项目验收评审人员。

1.2 项目背景

随着互联网技术的快速发展，社交网络已成为人们日常生活中不可或缺的组成部分。本项目为武汉大学计算机学院2026年小学期综合实践课程选题，目标是在6天内开发一个可在Web端运行的社交网络平台原型系统，聚焦动态分享、好友互动、个人资料管理等核心社交功能，探索实时通信技术在Web应用中的实践。

1.3 术语定义

术语	定义
动态	用户发布的图文内容，包括文字和图片，展示在首页信息流中
点赞	用户对某条动态表示喜欢的行为，每个用户对每条动态仅能点赞一次
评论	用户对某条动态发表文字评价，一条动态可有多条评论
好友	指双向互相关注、且好友请求已通过确认的用户关系
私信	用户与好友之间的一对一实时文字消息通信
信息流	按时间倒序排列的所有用户发布的动态列表
WebSocket	一种在单个TCP连接上进行全双工通信的协议，用于实现实时消息推送

第2章 系统总体描述

2.1 用户角色

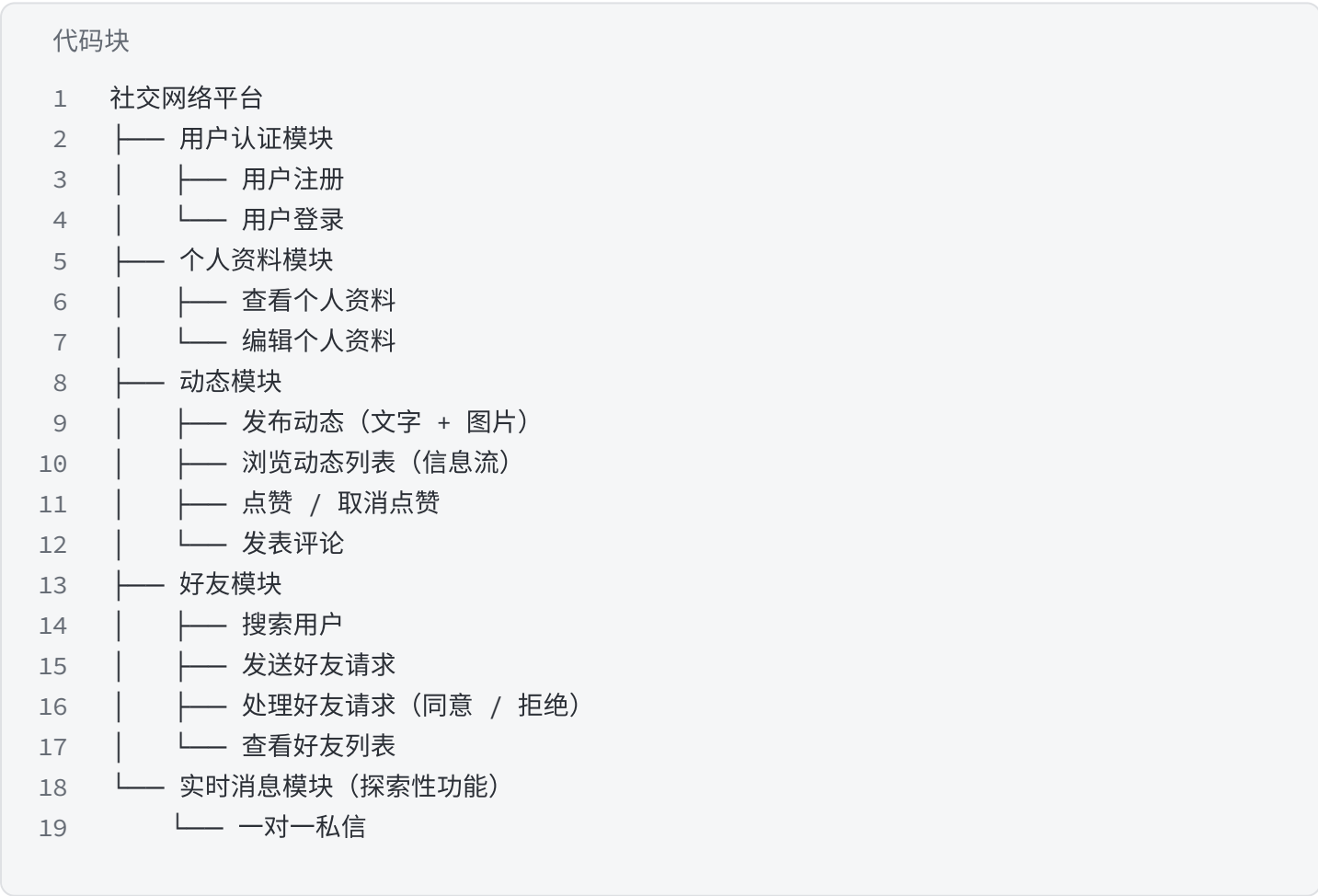
本系统涉及以下用户角色：

角色	描述	主要权限
普通用户	系统的核心使用者，已注册并登录	发布动态、浏览信息流、点赞评论、添加好友、私信聊天、编辑资料
游客	未注册或未登录的访问者	仅可查看登录页和注册页，无法使用任何社交功能

说明： 本项目面向C端个人用户，暂不设管理员角色。系统以用户自我管理为核心，用户可自主控制个人资料的可见性。

2.2 系统功能总览

社交网络平台的功能结构如下图所示：



2.3 业务需求与流程

本系统面向的主要用户群体是希望进行日常社交互动、分享生活内容的普通互联网用户。以下是本系统支持的四个核心用户场景，覆盖了从账号注册到深度社交互动的完整闭环。

2.3.1 场景一：用户注册与登录

场景描述：

小向是一名大学生，她听朋友说有一个新的社交平台可以分享日常、结交朋友。她打开浏览器访问平台的网页，在首页看到了清晰的登录和注册入口。

小向点击"注册"按钮，填写了用户名"xiangxiaoyu"和密码"12345678"，系统验证用户名未被占用后，提示"注册成功"，并自动跳转到登录页面。小向输入刚才注册的账号和密码，点击登录，系统验证通过后跳转到平台首页。

至此，小向正式成为该平台的用户，获得了发布动态、浏览内容、与他人互动等全部权限。

场景流程：

① 小向打开浏览器，访问社交网络平台网页 → ② 点击注册入口，填写用户名和密码 → ③ 系统验证通过，提示注册成功 → ④ 小向返回登录页，输入账号密码 → ⑤ 系统验证身份，登录成功，进入首页

涉及的用例： 用户注册、用户登录

2.3.2 场景二：发布动态与信息流浏览

场景描述：

小向登录后进入首页，看到其他用户发布的各种动态。她看到朋友小胡分享了一张日落照片，觉得很美，便点击了点赞按钮，并在评论区留言"太好看了！"。

小向自己也有一条好消息想分享——她今天收到了一份心仪的实习offer。她点击首页的"发布动态"按钮，在发布页输入文字"终于拿到心仪的实习offer了！开心！"，并上传了一张庆祝的照片。点击"发布"后，系统提示"发布成功"，她的动态出现在首页信息流的最上方，其他用户可以看到并互动。

场景流程：

① 小向在首页浏览动态列表 → ② 看到小胡的动态，点击点赞和发表评论 → ③ 小向点击发布动态入口 → ④ 输入文字并上传图片 → ⑤ 点击发布，系统保存动态 → ⑥ 动态出现在信息流顶部

涉及的用例： 浏览动态列表、点赞/取消点赞、发表评论、发布动态

2.3.3 场景三：建立好友关系

场景描述：

小向在浏览动态时，发现一个叫"小胡"的用户经常分享有趣的内容，她想和小胡成为好友，方便以后看到彼此的动态。

小向在搜索框中输入"小胡"，系统返回了匹配的用户列表。小向找到小胡的账号，点击"添加好友"按钮。系统向小胡发送了一条好友请求。

第二天，小胡登录平台后，在消息通知中看到了小向的好友请求。小胡查看了小向的个人主页，发现对方是一个喜欢分享生活的用户，便点击"同意"按钮。系统提示"你们已成为好友"。从此，两人在动态信息流中能更方便地关注对方的更新。

场景流程：

① 小向在搜索框中输入"小胡" → ② 系统返回用户列表 → ③ 小向点击"添加好友" → ④ 系统向小胡发送请求 → ⑤ 小胡登录后看到请求 → ⑥ 小胡点击"同意" → ⑦ 系统建立好友关系，双方成为好友

涉及的用例： 搜索用户、发送好友请求、处理好友请求

2.3.4 场景四：好友间的实时沟通（探索性功能）

场景描述：

小向和小胡已经成为好友。一天，小向想约小胡周末一起看电影，她打开好友列表，找到小胡，点击"发消息"按钮进入聊天界面。

小向输入"周末一起去看电影吗？"并点击发送。小胡正在线上，立刻收到了消息提醒，回复"好呀！什么时候？"两人通过平台的私信功能完成了这场实时对话。

说明： 该场景对应的私信功能为本次开发的探索性功能，团队将尽力实现，但如因时间或技术难度原因未能完成，将以其他核心功能的完整实现作为交付重点。

场景流程：

① 小向在好友列表中点击小胡 → ② 进入聊天界面 → ③ 输入消息并发送 → ④ 小胡实时收到消息并回复 → ⑤ 双方完成实时对话

涉及的用例： 发送私信、接收私信（探索性功能）

2.4 用户场景与功能模块对应关系

场景编号	场景名称	涉及功能
S1	用户注册与登录	用户注册、用户登录
S2	发布动态与互动	发布动态、浏览动态列表、点赞、评论
S3	建立好友关系	搜索用户、发送好友请求、处理好友请求
S4	好友间实时沟通	发送私信、接收私信

第3章 功能性需求

3.1 功能性需求列表

3.1.1 用户认证模块

--	--	--	--	--

需求编号	需求名称	描述	输入	输出
FR-01	用户注册	用户填写用户名和密码，系统验证后创建新账号	用户名、密码	注册成功/失败提示

业务约束：

- 用户名长度为3~20个字符，且唯一
- 密码长度不少于6位

3.1.2 个人资料模块

需求编号	需求名称	描述	输入	输出
FR-02	查看个人资料	用户查看自己的昵称、头像、简介等信息	用户ID	用户资料信息
FR-03	编辑个人资料	用户修改昵称、头像、性别、个人简介等信息	修改后的字段	更新成功提示

业务约束：

- 昵称长度不超过20个字符
- 个人简介长度不超过200个字符

3.1.3 动态模块

需求编号	需求名称	描述	输入	输出
FR-04	发布动态	用户输入文字并上传图片，发布一条新动态	文字内容、图片文件	发布成功/失败提示
FR-05	浏览动态列表	首页按时间倒序展示所有用户发布的动态	页码、每页条数	动态列表数据
FR-06	点赞/取消点赞	用户对某条动态点赞或取消点赞	动态ID	更新后的点赞数
FR-07	发表评论	用户对某条动态发表文字评论	动态ID、评论内容	评论成功提示

业务约束：

- 动态文字内容不超过1000字符

- 每次动态最多可上传9张图片
- 同一用户对同一条动态仅能点赞一次（再次点击为取消）
- 评论内容不超过200字符

3.1.4 好友模块

需求编号	需求名称	描述	输入	输出
FR-08	搜索用户	按用户名关键词搜索其他用户	关键词	用户列表
FR-09	发送好友请求	向其他用户发送好友申请	对方用户ID	请求发送成功提示
FR-10	处理好友请求	用户查看收到的请求并选择同意或拒绝	请求ID、操作类型	操作成功提示
FR-11	查看好友列表	查看已通过的所有好友	用户ID	好友列表

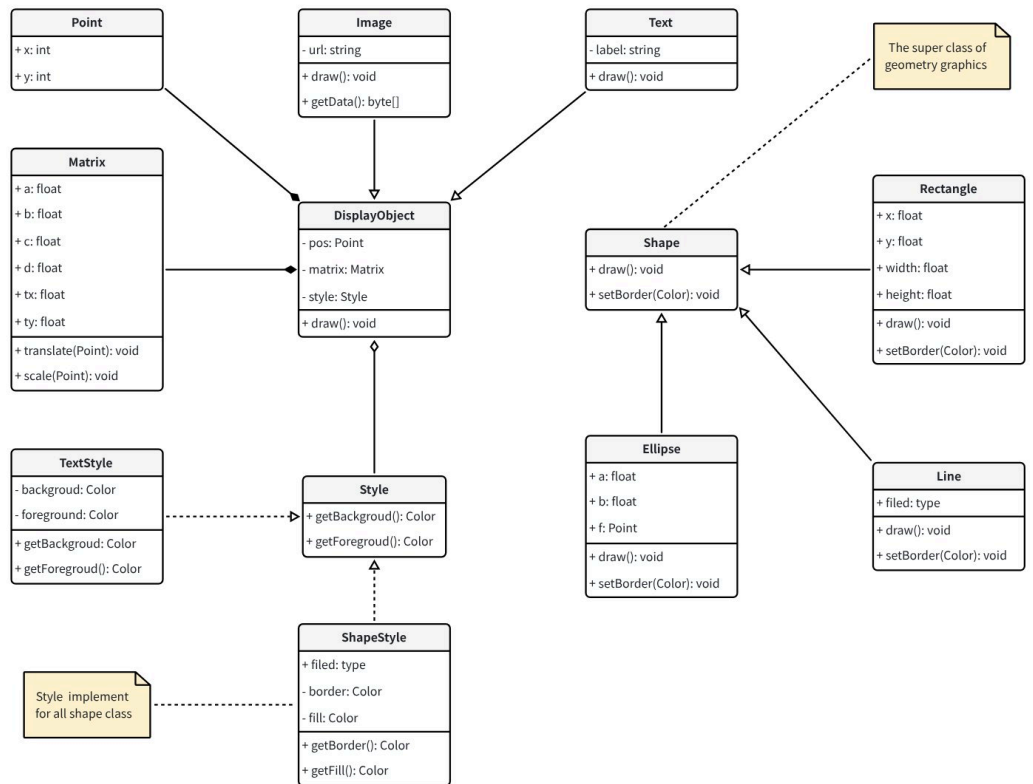
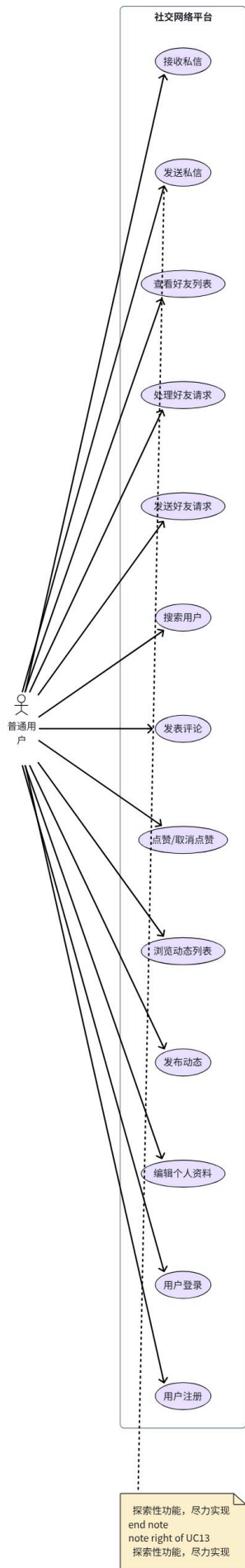
3.1.5 实时消息模块（探索性功能，尽力实现）

需求编号	需求名称	描述	输入	输出
FR-12	发送私信	用户向好友发送实时文字消息	接收方ID、消息内容	消息发送成功
FR-13	接收私信	用户实时接收好友发来的消息	—	消息内容实时展示

3.2 软件需求的用例模型

3.2.1 系统用例图

下图为社交网络平台的UML用例图，从外部用户视角描述了系统的主要功能：



3.2.2 用例详细说明

以核心用例“发布动态”为例进行详细说明：

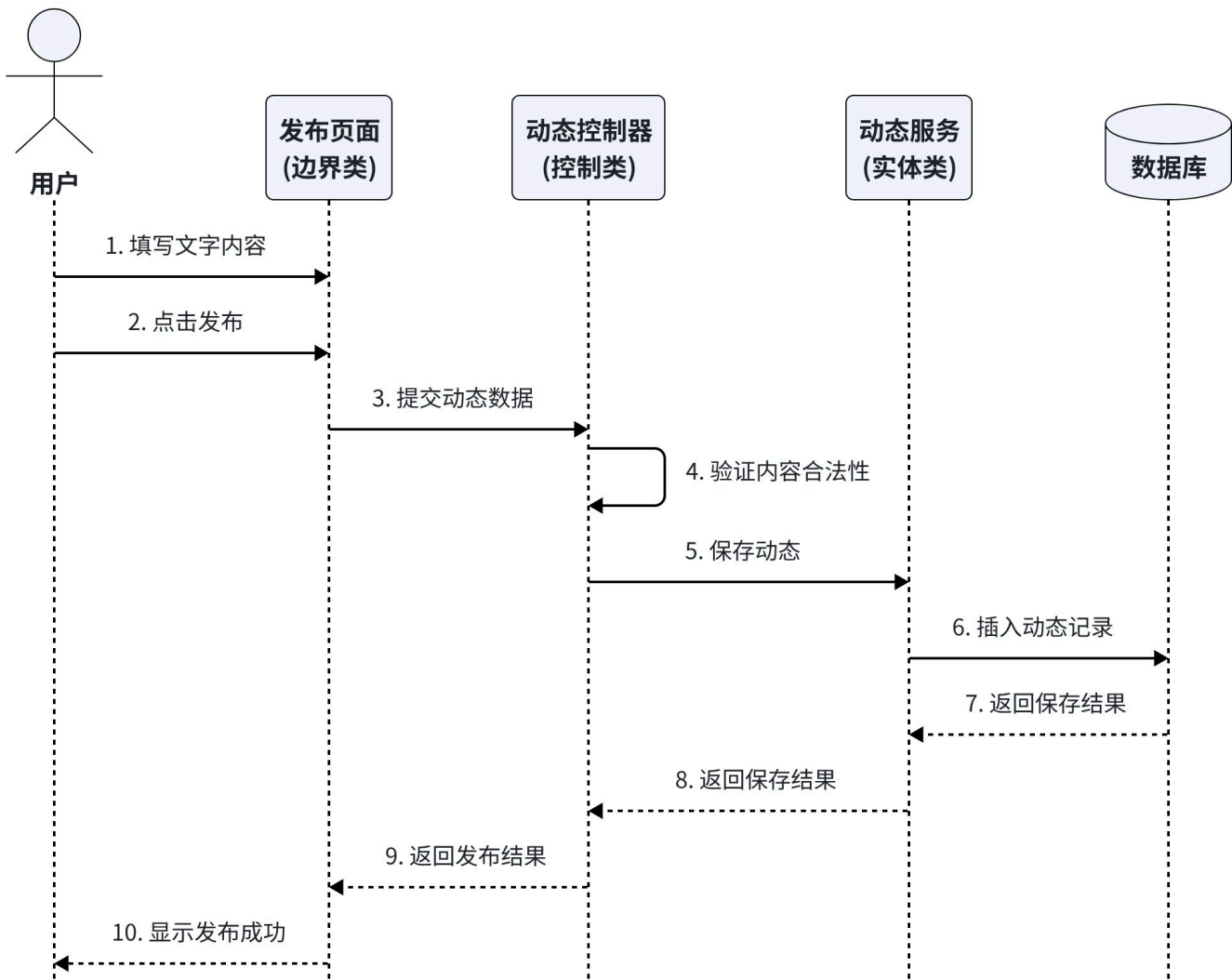
项目	内容
用例名称	发布动态
用例标识	UC-01
主要执行者	普通用户
前置条件	用户已登录系统
后置条件	动态成功保存至数据库，出现在首页信息流中
基本事件流	1. 用户进入发布动态页面 2. 用户输入文字内容（最多1000字符） 3. 用户选择上传图片（最多9张） 4. 用户点击“发布”按钮 5. 系统验证内容合法性 6. 系统保存动态至数据库 7. 系统提示“发布成功”，跳转至首页
备选事件流	5a. 文字内容为空 → 系统提示“请输入内容” 5b. 图片格式不支持 → 系统提示“仅支持JPG/PNG格式”

3.3 软件需求的分析模型

3.3.1 用例的交互模型——顺序图

以“发布动态”用例为例，绘制对象间交互的顺序图：

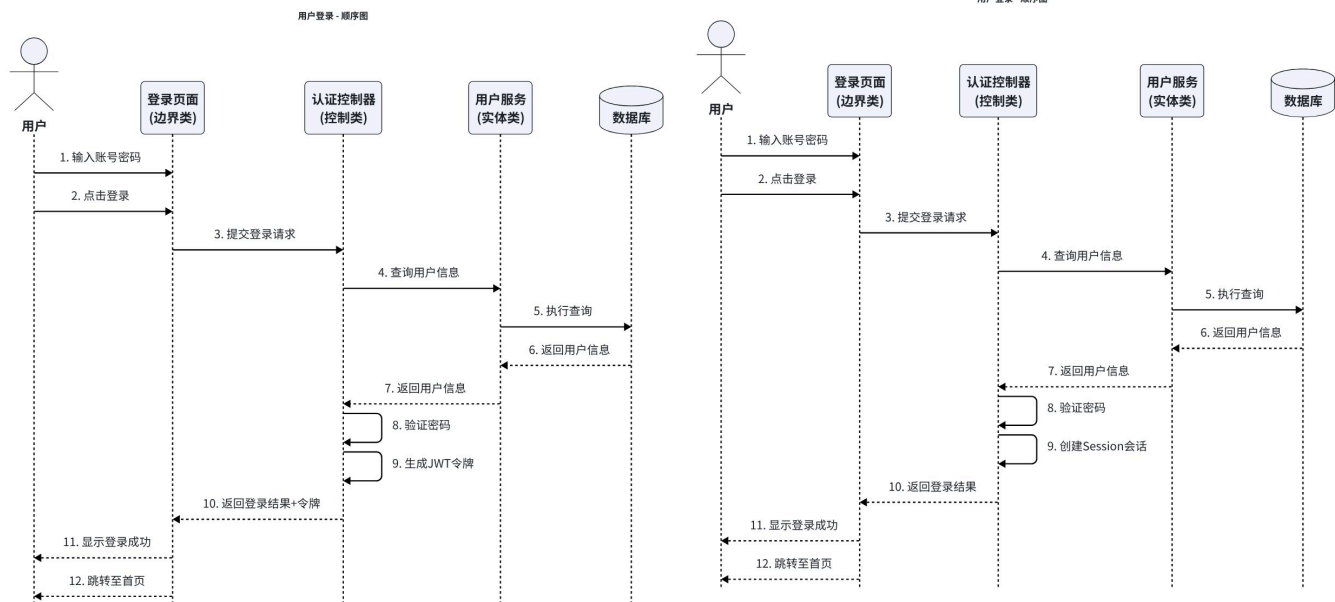
发布动态 - 顺序图



顺序图说明：

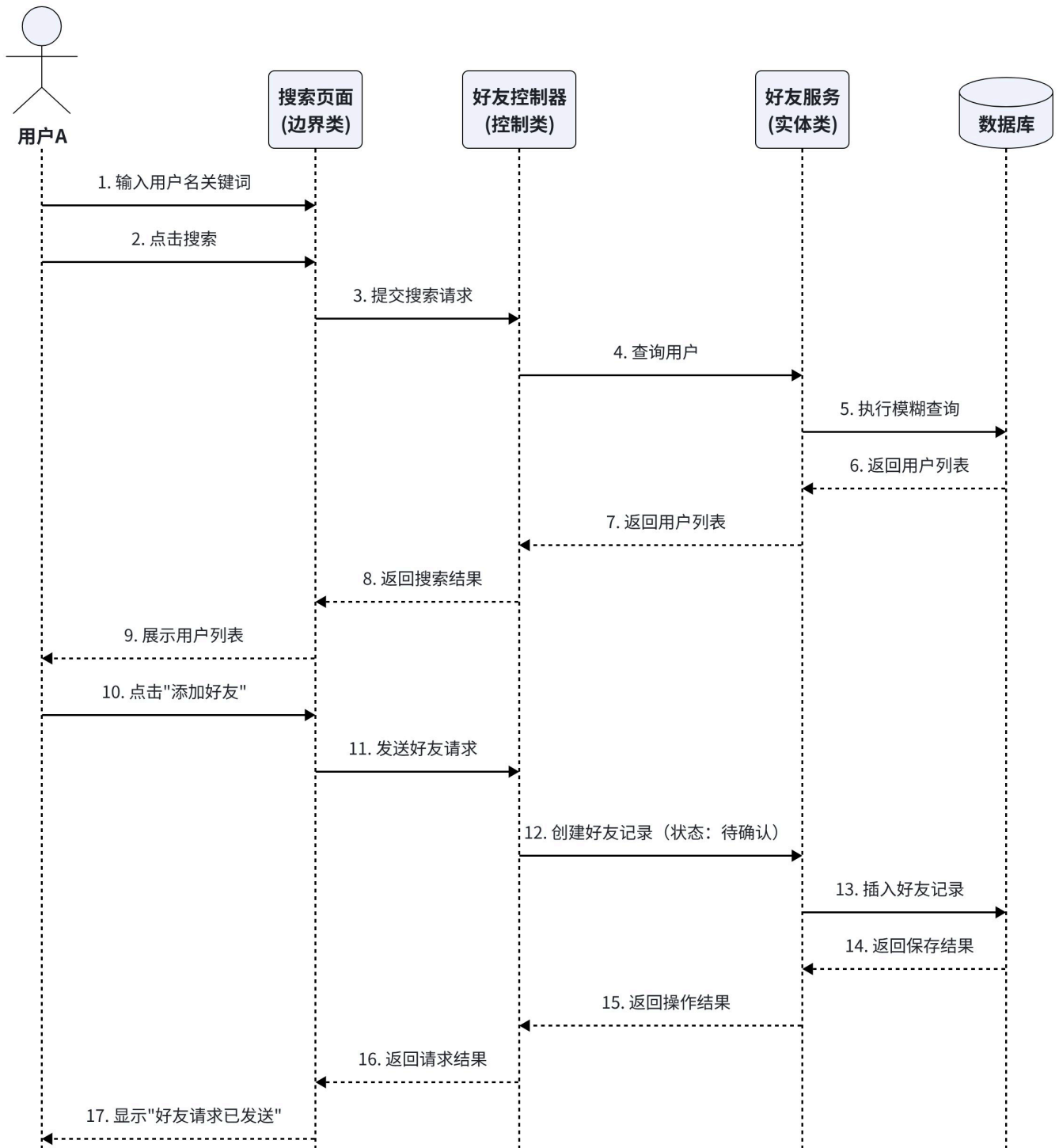
- **边界类（发布页面）**：负责接收用户输入的文字和图片，将用户操作转化为系统请求
- **控制类（动态控制器）**：负责协调整个发布流程，进行内容验证，调用实体类完成数据持久化
- **实体类（动态服务）**：负责将动态数据保存至数据库，并返回操作结果

以“用户登录”用例为例：



以“添加好友”用例为例：

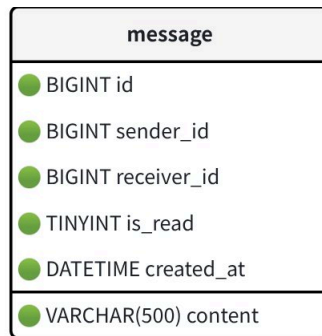
添加好友 - 顺序图



3.4 数据建模——ER图与数据字典

3.4.1 数据库概念设计（ER图）

本系统涉及以下核心数据实体及其关系：



n

实体关系说明：

关系	类型	说明
User → Post	一对多	一个用户可以发布多条动态
User → Comment	一对多	一个用户可以发表多条评论
Post → Comment	一对多	一条动态可以有多条评论
User → Like	一对多	一个用户可以点赞多条动态
Post → Like	一对多	一条动态可以被多个用户点赞
User → Friend	一对多	一个用户可以有多条好友记录
User → Message(发送)	一对多	一个用户可以发送多条私信
User → Message(接收)	一对多	一个用户可以接收多条私信

3.4.2 数据字典

(1) 用户表 (user)

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	用户唯一标识
username	VARCHAR(20)	NOT NULL, UNIQUE	用户名，用于登录
password	VARCHAR(100)	NOT NULL	密码
nickname	VARCHAR(20)	DEFAULT NULL	用户昵称
avatar	VARCHAR(255)	DEFAULT NULL	头像图片URL
bio	VARCHAR(200)	DEFAULT NULL	个人简介
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	注册时间
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE	更新时间

(2) 动态表 (post)

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	动态唯一标识
user_id	BIGINT		发布者ID

		NOT NULL, FOREIGN KEY → user(id)	
content	VARCHAR(1000)	NOT NULL	文字内容
images	VARCHAR(2000)	DEFAULT NULL	图片URL列表（JSON数组或逗号分隔）
likes_count	INT	DEFAULT 0	点赞总数
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	发布时间
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE	更新时间

(3) 评论表 (comment)

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	评论唯一标识
post_id	BIGINT	NOT NULL, FOREIGN KEY → post(id)	所属动态ID
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	评论者ID
content	VARCHAR(200)	NOT NULL	评论内容
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	评论时间

(4) 点赞表 (like)

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	点赞记录唯一标识
post_id	BIGINT	NOT NULL, FOREIGN KEY → post(id)	被点赞动态ID
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	点赞用户ID
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	点赞时间

约束说明：(user_id, post_id) 组合唯一，确保同一用户对同一条动态只产生一条点赞记录。

(5) 好友关系表 (friend)

--	--	--	--

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	好友记录唯一标识
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	发起方用户ID
friend_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	接收方用户ID
status	TINYINT	NOT NULL, DEFAULT 0	状态：0待确认、1已确认、2已拒绝
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	请求发起时间
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE	状态更新时间

约束说明： (user_id, friend_id) 组合唯一，确保不会重复发送好友请求。

(6) 私信表（message）——探索性功能

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	消息唯一标识
sender_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	发送者ID
receiver_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	接收者ID
content	VARCHAR(500)	NOT NULL	消息内容
is_read	TINYINT	DEFAULT 0	是否已读：0未读、1已读
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	发送时间

第4章 非功能性需求

4.1 性能需求

需求编号	需求描述
NFR-P-01	页面加载时间在正常网络环境下不超过3秒
NFR-P-02	API接口响应时间在正常数据量下不超过500毫秒
NFR-P-03	系统支持同时在线用户数不少于10人（课程演示环境）

4.2 可用性需求

需求编号	需求描述
NFR-U-01	系统界面简洁直观，符合移动端和PC端的操作习惯
NFR-U-02	操作错误时给出明确的提示信息，如“用户名不能为空”、“密码错误”等
NFR-U-03	所有按钮和可交互元素在点击时给予视觉反馈

4.3 安全性需求

需求编号	需求描述
NFR-S-01	用户密码加密存储，不存储明文密码
NFR-S-02	除注册和登录接口外，所有API接口均需验证JWT令牌的有效性
NFR-S-03	JWT令牌设置有效期，过期后需重新登录
NFR-S-04	好友关系需双方确认（申请→待确认→已确认），防止单向骚扰

4.4 兼容性需求

需求编号	需求描述
NFR-C-01	支持主流桌面浏览器：Chrome 90+、Edge 90+
NFR-C-02	支持移动端浏览器（响应式设计适配手机屏幕）
NFR-C-03	页面布局采用响应式设计，在PC和手机上都可获得良好的浏览体验

4.5 系统运行环境与开发环境

4.5.1 开发环境

项目	规格
操作系统	Windows 11
开发工具	Visual Studio Code（含Java和Vue插件）

后端框架	Spring Boot
前端框架	Vue 3 + Vite
JDK版本	OpenJDK 21
Node.js版本	18.04
数据库	MySQL 8.0.46
版本控制	Git + GitHub

4.5.2 运行环境

项目	规格
服务器操作系统	Windows / Linux（开发阶段本地运行）
数据库	MySQL 8.0
浏览器	Chrome 90+ / Edge 90+ / Safari 最新版
网络	可访问互联网（用于加载前端CDN资源）

说明： 本项目为课程实践项目，开发阶段在本地环境运行和演示，暂不部署至云服务器。

第5章 接口需求

5.1 前端页面路由

路由路径	页面	访问权限
/login	登录页	游客
/register	注册页	游客
/home	首页（信息流）	已登录用户
/publish	发布动态页	已登录用户
/profile	个人资料页	已登录用户
/profile/edit	编辑个人资料页	已登录用户
/friends	好友列表页	已登录用户

/friends/search	搜索用户页	已登录用户
/chat	私信聊天页（探索性）	已登录用户

5.2 后端API接口概览

接口	方法	说明	认证要求
/api/auth/register	POST	用户注册	无
/api/auth/login	POST	用户登录	无
/api/users/profile	GET	获取当前用户资料	需JWT
/api/users/profile	PUT	更新用户资料	需JWT
/api/posts	GET	获取动态列表（分页）	需JWT
/api/posts	POST	发布动态	需JWT
/api/posts/{id}/like	POST	点赞/取消点赞	需JWT
/api/posts/{id}/comments	POST	发表评论	需JWT
/api/users/search	GET	搜索用户	需JWT
/api/friends/requests	POST	发送好友请求	需JWT
/api/friends/requests	GET	获取好友请求列表	需JWT
/api/friends/requests/{id}	PUT	处理好友请求（同意/拒绝）	需JWT
/api/friends	GET	获取好友列表	需JWT
/ws/chat	WebSocket	实时聊天连接（探索性）	需JWT